



LambdaProof Policy Rules Specification

Overview

Policy rules in LambdaProof define the conditions under which an agent may perform actions. Rather than hardcoding permissions, policies express constraints declaratively—allowing the system to evaluate intent against context, evidence, and ethical boundaries at runtime.

Policies are **public specifications**. How they're evaluated (risk classification, optimization, evidence weighting) is proprietary. But the rule format itself is transparent so that:

1. Auditors can verify what constraints exist
2. Builders can write compatible policies
3. Users can understand what governs agent behavior

Rule Format

Basic Structure

A policy rule consists of four components:

```
WHEN <condition>  
REQUIRE <evidence>  
THEN <decision>
```

WITH <obligations>

Component	Purpose	Example
WHEN	Matches intent against tool, action, parameters	<code>tool = "email" AND action = "send"</code>
REQUIRE	Evidence needed before decision	<code>mfa_verified = true</code>
THEN	The authorization decision	ALLOW, DENY, ESCALATE
WITH	Post-decision obligations	<code>log_full_content = true</code>

Canonical JSON Representation

```
{
  "rule_id": "email-send-external",
  "version": "1.0",
  "when": {
    "tool": "email",
    "action": "send",
    "params": {
      "to": { "not_matches": "@internal.company.com$" }
    }
  },
  "require": {
    "evidence": ["session_authenticated", "mfa_verified"],
    "context": {
      "time_of_day": { "between": ["06:00", "22:00"] }
    }
  },
  "then": "ALLOW",
  "with": {
    "proof_level": "ATTESTATION",
    "log": ["recipient", "subject", "timestamp"],
    "redact": ["body"]
  }
}
```

Condition Matching (WHEN)

The when clause matches against the intent structure. Supported operators:

Operator	Meaning	Example
=	Exact match	action = "send"
in	One of set	action in ["send", "forward"]
matches	Regex pattern	to matches "@gmail.com\$"
not_matches	Regex negation	to not_matches "@internal.com\$"
>, <, >=, <=	Numeric comparison	amount > 1000
exists	Parameter present	attachment exists
not_exists	Parameter absent	cc not_exists

Compound Conditions

```
{
  "when": {
    "AND": [
      { "tool": "payments" },
      { "action": "transfer" },
      { "OR": [
        { "params.amount": { ">": 10000 } },
        { "params.to_country": { "in": ["sanctioned_list"] } }
      ]}
    ]
  }
}
```

Evidence Requirements (REQUIRE)

Evidence represents proof that certain conditions hold. Evidence is **referenced by ID**, not embedded—the actual validation happens server-side.

Evidence Types

Type	What It Proves	Example Reference
session_authenticated	User has active session	evidence:sess_abc123
mfa_verified	Multi-factor auth completed	evidence:mfa_xyz789
user_confirmed	Explicit user approval	evidence:confirm_456
manager_approved	Human escalation completed	evidence:approval_789
context_verified	Contextual check passed	evidence:ctx_geo_us
zk_proof	Zero-knowledge proof valid	evidence:zk_age_over_18

Evidence Freshness

Evidence can have freshness requirements:

```
{
  "require": {
    "evidence": [
      { "type": "mfa_verified", "max_age_seconds": 300 },
      { "type": "session_authenticated", "max_age_seconds": 3600 }
    ]
  }
}
```

Decisions (THEN)

Three possible decisions:

Decision	Meaning	When Used
ALLOW	Action may proceed	All conditions and evidence satisfied

DENY	Action is blocked	Conditions fail or evidence insufficient
ESCALATE	Requires human review	High-risk or ambiguous situations

Decision with Conditions

```
{
  "then": {
    "decision": "ALLOW",
    "if_evidence_missing": "ESCALATE",
    "if_context_invalid": "DENY"
  }
}
```

Obligations (WITH)

Obligations specify what must happen alongside or after the decision:

Obligation	Purpose	Example
proof_level	Cryptographic proof required	"NONE", "ATTESTATION", "ZK"
log	Fields to audit log	["recipient", "amount"]
redact	Fields to exclude from logs	["body", "password"]
notify	Alert recipients	["security@company.com"]
expire_seconds	Token validity window	300
max_uses	Token use limit	1

Example Policies

Low-Risk: Web Navigation

```
{
  "rule_id": "web-navigate-allowed-domains",
  "description": "Allow navigation to pre-approved domains",
  "when": {
    "tool": "web",
    "action": "navigate",
    "params.url": { "matches": "^https://(google|github|stackoverflow)\\.com" }
  },
  "require": {
    "evidence": ["session_authenticated"]
  },
  "then": "ALLOW",
  "with": {
    "proof_level": "NONE",
    "log": ["url", "timestamp"]
  }
}
```

Behavior: Agent can browse approved sites with minimal friction. Only session auth required, no cryptographic proof, basic logging.

Medium-Risk: Email to External Recipients

```
{
  "rule_id": "email-send-external",
  "description": "Send email to external addresses requires MFA",
  "when": {
    "tool": "email",
    "action": "send",
    "params.to": { "not_matches": "@company\\.com$" }
  },
  "require": {
    "evidence": [
      "session_authenticated",
      { "type": "mfa_verified", "max_age_seconds": 600 }
    ]
  },
  "then": "ALLOW",
}
```

```

"with": {
  "proof_level": "ATTESTATION",
  "log": ["to", "subject", "timestamp", "attachment_count"],
  "redact": ["body"],
  "expire_seconds": 120
}
}

```

Behavior: External emails require recent MFA. Attestation proof generated. Body content redacted from logs. Token expires in 2 minutes.

High-Risk: Financial Transfer

```

{
  "rule_id": "payment-transfer-large",
  "description": "Large transfers require explicit confirmation and manager approval",
  "when": {
    "tool": "payments",
    "action": "transfer",
    "params.amount": { ">": 10000 }
  },
  "require": {
    "evidence": [
      "session_authenticated",
      { "type": "mfa_verified", "max_age_seconds": 300 },
      { "type": "user_confirmed", "confirmation_text": "I approve this transfer of
${amount} to ${recipient}" },
      { "type": "manager_approved", "timeout_hours": 24 }
    ],
    "context": {
      "geo_location": { "in": ["US", "CA", "GB"] },
      "time_of_day": { "between": ["09:00", "17:00"] },
      "device_trusted": true
    }
  },
  "then": {
    "decision": "ALLOW",
    "if_evidence_missing": "ESCALATE"
  }
}

```

```

    },
    "with": {
      "proof_level": "ZK",
      "log": ["amount", "recipient", "timestamp", "approver"],
      "notify": ["finance@company.com", "security@company.com"],
      "expire_seconds": 60,
      "max_uses": 1
    }
  }
}

```

Behavior: Large transfers require full evidence chain: session + MFA + explicit user confirmation + manager approval. Contextual checks for location, time, and device. Zero-knowledge proof generated. Single-use token expires in 60 seconds. Finance and security notified.

Deny Rule: Sanctioned Destinations

```

{
  "rule_id": "payment-deny-sanctioned",
  "description": "Block all transfers to sanctioned countries",
  "priority": 1000,
  "when": {
    "tool": "payments",
    "action": "transfer",
    "params.to_country": { "in": ["$SANCTIONED_COUNTRIES"] }
  },
  "require": {},
  "then": "DENY",
  "with": {
    "log": ["amount", "to_country", "timestamp"],
    "notify": ["compliance@company.com"],
    "deny_reason": "Transfer destination is on sanctioned list"
  }
}

```

Behavior: Unconditional deny. High priority ensures this rule evaluates first. Compliance team notified on every attempt.

Escalation Rule: Unusual Patterns

```
{
  "rule_id": "email-escalate-bulk",
  "description": "Bulk email sending requires human review",
  "when": {
    "tool": "email",
    "action": "send",
    "session_context.emails_sent_last_hour": { ">": 50 }
  },
  "require": {
    "evidence": ["session_authenticated", "mfa_verified"]
  },
  "then": "ESCALATE",
  "with": {
    "escalation_queue": "security-review",
    "escalation_timeout_hours": 4,
    "escalation_context": ["recent_email_count", "recipient_domains"],
    "if_timeout": "DENY"
  }
}
```

Behavior: When an agent tries to send many emails quickly, action pauses for human review. Security team has 4 hours to approve/deny. Auto-denies if no response.

Rule Priority and Evaluation

Rules are evaluated in priority order (higher number = higher priority):

```
{
  "policy_bundle": {
    "version": "1.0",
    "rules": [
      { "rule_id": "payment-deny-sanctioned", "priority": 1000 },
      { "rule_id": "payment-transfer-large", "priority": 100 },
      { "rule_id": "payment-transfer-default", "priority": 10 }
    ]
  }
}
```

```
    ],  
    "default_decision": "DENY"  
  }  
}
```

Evaluation logic:

1. Find all rules where `when` clause matches the intent
 2. Sort matching rules by priority (descending)
 3. Evaluate highest-priority rule first
 4. If evidence requirements met → return that decision
 5. If no rules match → return `default_decision`
-

CSL Integration

Policies can reference CSL (Constitutional Safety Language) operators for ethical constraints:

```
{  
  "rule_id": "action-csl-check",  
  "when": {  
    "tool": "*",  
    "action": "*"  
  },  
  "require": {  
    "csl_validation": {  
      "operators": ["CAUSE", "CONSENT", "SILENCE"],  
      "must_satisfy": "ALL"  
    }  
  },  
  "then": {  
    "decision": "ALLOW",  
    "if_csl_fails": "DENY"  
  }  
}
```

CSL operators are evaluated server-side. The policy simply declares that CSL validation is required.

Policy Bundles

Policies are grouped into bundles for deployment:

```
{
  "bundle_id": "enterprise-standard-v2",
  "version": "2.0.0",
  "effective_date": "2026-01-15T00:00:00Z",
  "rules": [
    { "$ref": "rules/email-send-external.json" },
    { "$ref": "rules/payment-transfer-large.json" },
    { "$ref": "rules/payment-deny-sanctioned.json" }
  ],
  "defaults": {
    "proof_level": "ATTESTATION",
    "log_all_decisions": true,
    "default_decision": "DENY"
  },
  "metadata": {
    "author": "security-team",
    "approved_by": "compliance-officer",
    "audit_hash": "sha256:abc123..."
  }
}
```

What's Public vs. Proprietary

Public (This Document)	Proprietary (Server-Side)
Rule format specification	Risk classification of intents
Condition matching syntax	Policy optimization algorithms
Evidence type definitions	Evidence weighting heuristics

Decision and obligation types	CSL commutation solver
Example policies	Token signing implementation
Bundle structure	Evaluation performance optimizations

The format is transparent. The intelligence is proprietary.

Next Steps

- **For Builders:** Use this format to write policies for your domain
 - **For Auditors:** Review deployed policy bundles against this spec
 - **For Researchers:** Propose extensions via the contribution process
-

This specification is part of the Λ Proof Constitutional Core and is published under CC0 (public domain).

Quick Thoughts on the Draft

This document achieves several goals:

1. **Transparency** - Anyone can understand how policies work
2. **Interoperability** - Third parties can write compatible policies
3. **Auditability** - Auditors can verify what rules govern agent behavior
4. **Strategic opacity preserved** - HOW rules are evaluated (risk scoring, optimization) stays secret

Want me to adjust the tone, add more examples, expand any section, or add a specific use case (healthcare, finance, supply chain)?

**

1. Milestone-Vision-Timeframe.csv
 - a. Λ proof Healthcare Protocols V0.1.pdf
 - b. Λ dapps On Web4 - Developer Overview.pdf
2. Building a Λ Dapp_ A Developer's Implementation Guide.pdf
3. MTPI_Instructions.pdf
4. MTPI RootContract.pdf
5. MTPI_RootContract_Project.pdf
6. Patent Search Results.pdf
7. The Λ Proof Architecture_ A Comprehensive Analysis of its Implications for Web4.pdf
8. The Λ Proof Architecture_ Web4.pdf
9. Verifiable, Privacy-Preserving Applications.pdf
10. Lets further develop the SAFE_ awesome—let's turn.pdf
11. Λ Proof.pdf
12. Ξ Constitution.pdf
13. Λ^P -Archivum.pdf
 - c. Λ proof $\times$ Mtpi Lawful-by-design Integration — Implementation Plan.pdf
14. 2. Λ RootContract II & III - MTPI - NODE.js.pdf
 - d. Λ proof Monorepo — Plain-language Overview V0.pdf
 - e. Λ -trace Overview.pdf
 - f. Λ proof Banking Protocols.pdf